

claude-windows / 02b9e9c9-3806-45ca-9b24-2ecc35a81efd

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\subagents\workflows\wf_27df53c8-8e0\agent-a7e80fd03970de2de.jsonl`
- SHA-256: `e7470af73f98c25700af92b9db7872e708b328b0a3953322f1c98917b19cb962`
- Source modified: `2026-06-10T16:24:20+00:00`
- Imported at: `2026-07-05T16:48:21+00:00`
- project: `wf_27df53c8-8e0`
- session_id: `02b9e9c9-3806-45ca-9b24-2ecc35a81efd`
```

Transcript

1. user (2026-06-10T16:22:08.040Z)

You are an adversarial verifier. A code reviewer audited the repo at C:/Users/avidu/Projects/muneem and reported this finding:

TITLE: Key rotation overwrites the only copy of the current key before the file is rekeyed; rollback misses realistic failures
SEVERITY CLAIMED: critical
FILE: src/muneem/cli.py (line 438-446)
DESCRIPTION: `db rotate-key` writes the fresh key into the keychain FIRST (`db_key.set_key(fresh)`), destroying the only durable copy of the current key (it survives only in process memory as `current`), and only then rekeys the file. Two realistic failure modes permanently lock the ledger: (1) a crash/power loss/Ctrl-C between `set_key(fresh)` and a completed `PRAGMA rekey` leaves `keychain=fresh`, `file=old-key`, old key gone; (2) the rollback `except` only catches `dbmod.EncryptedDatabaseError`, but `rekey_database` (`src/muneem/db.py:274-279`) can raise `sqlcipher3.OperationalError` (e.g. 'database is locked' ? very plausible on Windows with another process/AV holding the file) or any other `sqlcipher3` error from `PRAGMA rekey`, which escapes uncaught, skips the rollback, and exits with `keychain=fresh / file=old`. Unless the user happens to have a recovery file (which wraps the OLD key), the entire financial ledger and all same-key backups are unrecoverable. The inline comment ('Keychain-first with rollback: if the file rekey fails, the keychain still matches the file') asserts the opposite of what the code does ? keychain-first guarantees a mismatch window.
EVIDENCE QUOTED: # Keychain-first with rollback: if the file rekey fails, the keychain still matches the file.

```
db_key.set_key(fresh)
try:
    dbmod.rekey_database(db_path, current, fresh)
except dbmod.EncryptedDatabaseError as exc:
    db_key.set_key(current) # roll back; the file is still under the old key
```

SUGGESTED FIX: Never have a moment where only one key exists in durable storage: write the fresh key under a secondary keychain name (e.g. 'db-key-pending') first, rekey the file, then promote it to 'db-key' and delete the pending entry; on open failure, fall back to trying the pending key. At minimum: rekey the file BEFORE touching the 'db-key' entry, broaden the except to catch all exceptions (re-raising after rollback), and stash the old key under 'db-key-prev' before overwriting.

Your job is to try to REFUTE it. Your lens: correctness: trace the actual code path the finding describes and check the claimed behavior is what the code really does (read the file and its callers/tests).

Read the actual code in the repo (and tests/config where relevant). If after reading you cannot confirm the problem is real, set `isReal=false`. If it is real but over/under-stated, set `adjustedSeverity` accordingly. Be strict: plausible-sounding but unconfirmed = refuted.

SENSITIVE-DATA RULES (mandatory, highest priority):

- testdata/aubank-statements/, testdata/axis-statements/, testdata/hdfc-statements/, testdata/snapshots/, and testdata/credentials/ in the muneem repo contain the user's REAL financial documents and a real OAuth client file. Do NOT open/read the PDFs. You MAY check file

existence and git-tracking status (git ls-files, git log --all -- <path>).

- If you must inspect testdata/snapshots/*.json or testdata/credentials/*.json to assess sensitivity, read at most the JSON key structure and NEVER quote values (no account numbers, balances, person names, client secrets, transaction amounts) in your output.
- Never include sensitive values in your findings. Refer to them abstractly ("a real account number", "a client_secret value").
- Do not use WebSearch/WebFetch and do not send any repo content to any external service.

Structured output only.

2. user (2026-06-10T16:22:13.387Z)

```
380         else:
381             state = "encrypted (SQLCipher)"
382             typer.echo(f"ledger : {db_path}")
383             typer.echo(f"state  : {state}")
384             typer.echo(f"key    : {key_state}")
385
386
387     @db_app.command("encrypt")
388     def db_encrypt(
389         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
390 dir)."),
391     ) -> None:
392         """Migrate an existing *plaintext* ledger to an encrypted one (SQLCipher), in
393 place."""
394         from muneem import db as dbmod
395         from muneem.db_key import get_or_create_key
396
397         db_path = db or load_config().db_path
398         if not Path(db_path).exists():
399             typer.echo(
400                 f"nothing to encrypt: {db_path} does not exist (a new ledger is born
401 encrypted)."
402             )
403             return
404         if not dbmod.is_plaintext_sqlite(db_path):
405             typer.echo(
406                 f"{Path(db_path).name} is already encrypted (or not a SQLite file); nothing
407 to do."
408             )
409             return
410         n = dbmod.encrypt_database(db_path, get_or_create_key())
411         typer.echo(
412             f"encrypted {Path(db_path).name}: {n} table(s) migrated. A plaintext backup
413 remains at "
414             f"{Path(db_path).name}.plaintext.bak ? verify, then securely delete it."
415         )
416
417
418     def _require_encrypted_ledger(db_path: Path) -> str:
419         """Return the ledger's keychain key, or exit with a clear message if it can't be
420 used."""
421         from muneem import db as dbmod
422         from muneem.db_key import peek_key
423
424         if not Path(db_path).exists() or dbmod.is_plaintext_sqlite(db_path):
425             typer.echo("no encrypted ledger here (run 'muneem db encrypt' first).",
426 err=True)
427             raise typer.Exit(1)
428         key = peek_key()
429         if not key:
430             typer.echo("no encryption key in the keychain for this ledger.", err=True)
431             raise typer.Exit(1)
432         return key
```

```

427
428     @db_app.command("rotate-key")
429     def db_rotate_key(
430         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
431 dir)."),
432     ) -> None:
433         """Rotate the ledger's encryption key (SQLCipher rekey) and update the keychain."""
434         from muneem import db as dbmod
435         from muneem import db_key
436
437         db_path = db or load_config().db_path
438         current = _require_encrypted_ledger(db_path)
439         fresh = db_key.new_key()
440         # Keychain-first with rollback: if the file rekey fails, the keychain still matches
441         the file.
442         db_key.set_key(fresh)
443         try:
444             dbmod.rekey_database(db_path, current, fresh)
445         except dbmod.EncryptedDatabaseError as exc:
446             db_key.set_key(current) # roll back; the file is still under the old key
447             typer.echo(str(exc), err=True)
448             raise typer.Exit(1) from exc
449         typer.echo(f"rotated the encryption key for {Path(db_path).name}.")
450
451     @db_app.command("backup")
452     def db_backup(
453         dest: Path = typer.Argument(..., help="Destination path for the encrypted backup
454 copy."),
455         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
456 dir)."),
457     ) -> None:
458         """Write an encrypted online-backup copy of the ledger (consistent even with WAL
459 active)."""
460         from muneem import db as dbmod
461
462         db_path = db or load_config().db_path
463         key = _require_encrypted_ledger(db_path)
464         try:
465             dbmod.backup_database(db_path, dest, key)
466         except dbmod.EncryptedDatabaseError as exc:
467             typer.echo(str(exc), err=True)
468             raise typer.Exit(1) from exc
469         typer.echo(f"backed up {Path(db_path).name} to {dest} (encrypted).")
470
471     @db_app.command("setup-recovery")
472     def db_setup_recovery(
473         out: Path | None = typer.Option(
474             None, help="Recovery file path (default: <data dir>/muneem-recovery.json)."
475         ),
476     ) -> None:
477         """Create a passphrase-wrapped recovery file for the ledger's encryption key.
478
479 Lets you restore the key (and thus the ledger + backups) if the OS keychain is ever
480 lost. The
481 file holds only the *wrapped* key ? useless without the passphrase.
482 """
483         import getpass
484
485         from muneem import recovery
486         from muneem.db_key import peek_key
487
488         key = peek_key()
489         if not key:

```

```

486         typer.echo("no encryption key in the keychain yet (create the ledger first).",
err=True)
487         raise typer.Exit(1)
488     out_path = out or (load_config().data_dir / "muneem-recovery.json")
489     passphrase = getpass.getpass("New recovery passphrase: ")

```

3. user (2026-06-10T16:22:14.745Z)

```

230         try:
231             src_tables = src.execute(
232                 "SELECT count(*) FROM sqlite_master WHERE type = 'table'"
233             ).fetchone()[0]
234             target = str(enc_tmp).replace("\\", "/") # SQL string literal: forward
slashes
235             src.execute(f"ATTACH DATABASE '{target}' AS encrypted KEY \x'{key}'")
236             src.execute("SELECT sqlcipher_export('encrypted')")
237             src.execute("DETACH DATABASE encrypted")
238         finally:
239             src.close()
240
241         # Verify the encrypted copy BEFORE the destructive swap: it must open with the
key, pass
242         # an integrity check, and carry every table across. (The plaintext .bak is kept
either way,
243         # so a failed check is recoverable ? but we refuse to swap in a bad file.)
244         with connect(str(enc_tmp), key=key) as check:
245             integrity = check.execute("PRAGMA integrity_check").fetchone()[0]
246             ntables = check.execute(
247                 "SELECT count(*) FROM sqlite_master WHERE type = 'table'"
248             ).fetchone()[0]
249         if integrity != "ok" or ntables != src_tables:
250             raise EncryptedDatabaseError(
251                 f"encryption migration verification failed for {path.name} "
252                 f"(integrity={integrity!r}, tables {ntables}/{src_tables}); original
left untouched"
253             )
254
255         backup = path.with_name(path.name + ".plaintext.bak")
256         backup.unlink(missing_ok=True)
257         path.rename(backup)
258         enc_tmp.rename(path) # consumed on success
259         _harden_permissions(str(path))
260     finally:
261         # Never leave a partial temp behind: no-op on success (already renamed), cleans
up on error.
262         enc_tmp.unlink(missing_ok=True)
263     return int(ntables)
264
265
266     def rekey_database(db_path: Path | str, current_key: str, new_key: str) -> None:
267         """Re-encrypt the ledger under ``new_key`` in place (SQLCipher ``PRAGMA rekey``).
268
269         Opens with ``current_key`` first (which verifies it), then rekeys. Raises
270         :class:`EncryptedDatabaseError` if ``current_key`` is wrong or the file is not an
encrypted
271         muneem ledger. The caller is responsible for updating the keychain entry to
``new_key`` only
272         after this returns successfully.
273         """
274         conn = _open_encrypted(str(db_path), current_key) # validates current_key
275         try:
276             conn.execute(f"PRAGMA rekey = \x'{new_key}'")
277             conn.commit()
278         finally:
279             conn.close()

```

```

280     _harden_permissions(str(db_path))
281
282
283     def backup_database(db_path: Path | str, dest: Path | str, key: str) -> None:
284         """Write an encrypted online-backup copy of the ledger to ``dest`` (same key).
285
286         Uses the SQLite Online Backup API for a consistent snapshot ? safe even with WAL
active, so no
287         naive file copy and no manual checkpoint needed. The destination is keyed with the
same key, so
288         it stays encrypted at rest. Raises :class:`EncryptedDatabaseError` if ``key`` is
wrong.
289         """
290         import sqlcipher3
291
292         dest_path = Path(dest)
293         dest_path.parent.mkdir(parents=True, exist_ok=True)
294         dest_path.unlink(missing_ok=True)
295         source = _open_encrypted(str(db_path), key) # validates key
296         try:
297             target = sqlcipher3.connect(str(dest_path))
298             try:
299                 target.execute(_raw_key_pragma(key)) # key the destination so the backup is
encrypted
300                 source.backup(target) # consistent snapshot of the live DB
301             finally:
302                 target.close()
303         finally:
304             source.close()
305         _harden_permissions(str(dest_path))
306
307
308     def init_db(conn: sqlite3.Connection) -> None:
309         """Bring the schema up to date via the migration runner (creates tables on a fresh
DB).
310
311         The current ``_SCHEMA`` is the frozen *baseline* (version ``SCHEMA_VERSION``);
future changes
312         are forward migrations in :mod:`muneem.migrations`, recorded in
``schema_migrations``.
313         """
314         from muneem import migrations
315
316         migrations.run_migrations(conn, baseline_sql=_SCHEMA,
baseline_version=SCHEMA_VERSION)
317
318
319     def get_document_by_sha(conn: sqlite3.Connection, sha256: str) -> sqlite3.Row | None:

```

4. user (2026-06-10T16:22:21.458Z)

```

1     """Local SQLite store: the ``documents`` and ``offers`` slice of the schema.
2
3     This is the minimal slice the promo spine needs (ROADMAP A.2). The full schema
4     (accounts, transactions, holdings, merchants, items) grows with later milestones.
5     ``documents.sha256`` is UNIQUE so re-ingesting the same file is a no-op.
6     ``offers.source`` records provenance ('text' = exact text layer, 'ocr' = approximate).
7     """
8
9     from __future__ import annotations
10
11     import os
12     import sqlite3
13     from collections.abc import Iterator
14     from contextlib import contextmanager

```

```

15     from pathlib import Path
16
17     from muneem.errors import EncryptedDatabaseError # re-exported so
db.EncryptedDatabaseError works
18
19     SCHEMA_VERSION = 4
20
21     # Unencrypted SQLite files start with this 16-byte magic header; an encrypted
(SQLCipher)
22     # file does not, so we can tell them apart without a key.
23     _SQLITE_MAGIC = b"SQLite format 3\x00"
24
25     _SCHEMA = ""
26     CREATE TABLE IF NOT EXISTS documents (
27         id            INTEGER PRIMARY KEY,
28         source        TEXT NOT NULL,
29         external_id   TEXT,
30         sender        TEXT,
31         subject       TEXT,
32         received_date TEXT,
33         doc_type      TEXT,
34         sha256        TEXT NOT NULL UNIQUE,
35         path          TEXT,
36         status        TEXT NOT NULL DEFAULT 'parsed',
37         created_at    TEXT NOT NULL DEFAULT (datetime('now'))
38     );
39
40     CREATE TABLE IF NOT EXISTS offers (
41         id            INTEGER PRIMARY KEY,
42         document_id   INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
43         merchant      TEXT,
44         promo_code    TEXT,
45         description   TEXT,
46         expiry_raw    TEXT,
47         expiry_type   TEXT,
48         source        TEXT NOT NULL DEFAULT 'text',
49         redeemed      INTEGER NOT NULL DEFAULT 0,
50         source_text   TEXT
51     );
52
53     CREATE INDEX IF NOT EXISTS idx_offers_document ON offers(document_id);
54
55     -- We use separate tables for each bank (e.g. axis_transactions, hdfc_transactions)
56     -- instead of a unified table to preserve raw schema fidelity of the source PDFs.
57     -- Normalization into a unified view layer is planned for a later phase
58     -- once all bank schemas are stabilized.
59     CREATE TABLE IF NOT EXISTS axis_transactions (
60         id INTEGER PRIMARY KEY,
61         document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
62         account_number TEXT,
63         txn_date TEXT,
64         value_date TEXT,
65         particulars TEXT,
66         debit REAL,
67         credit REAL,
68         balance REAL
69     );
70
71     CREATE TABLE IF NOT EXISTS axis_cc_transactions (
72         id INTEGER PRIMARY KEY,
73         document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
74         account_number TEXT,
75         txn_date TEXT,
76         particulars TEXT,
77         merchant_category TEXT,

```

```

78         amount REAL,
79         type TEXT
80     );
81
82     CREATE TABLE IF NOT EXISTS au_transactions (
83         id INTEGER PRIMARY KEY,
84         document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
85         account_number TEXT,
86         txn_date TEXT,
87         value_date TEXT,
88         particulars TEXT,
89         debit REAL,
90         credit REAL,
91         balance REAL
92     );
93
94     CREATE TABLE IF NOT EXISTS hdfc_transactions (
95         id INTEGER PRIMARY KEY,
96         document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
97         account_number TEXT,
98         txn_date TEXT,
99         value_date TEXT,
100        particulars TEXT,
101        ref TEXT,
102        withdrawal REAL,
103        deposit REAL,
104        closing_balance REAL
105    );
106    """
107
108
109    def _raw_key_pragma(key: str) -> str:
110        """The SQLCipher raw-key PRAGMA: use the 256-bit ``key`` (64 hex chars) directly, no
KDF. """
111        return f"PRAGMA key = \"x'{key}'\""
112
113
114    def is_plaintext_sqlite(db_path: Path | str) -> bool:
115        """True iff the file exists and begins with the *unencrypted* SQLite magic header.
116
117        Lets us refuse to silently treat a plaintext ledger as encrypted (and vice-versa)
118        without needing the key.
119        """
120        try:
121            with open(db_path, "rb") as fh:
122                return fh.read(16) == _SQLITE_MAGIC
123        except OSError:
124            return False
125
126
127    def _harden_permissions(path: str) -> None:
128        """Best-effort: restrict the DB file to its owner (0600). No-op on failure or
``:memory:``. """
129        if path == ":memory:":
130            return
131        try:
132            os.chmod(path, 0o600)
133        except OSError:
134            pass
135
136
137    def _open_encrypted(path: str, key: str) -> sqlite3.Connection:
138        """Open an encrypted connection via SQLCipher, verifying the key before returning.
139
140        A keyed open NEVER silently downgrades to plaintext: if the binding is missing, or

```

```

the
141     key is wrong (or the file is not an encrypted muneem ledger), this raises.
142     """
143     try:
144         import sqlcipher3
145     except ImportError as exc: # pragma: no cover - exercised only without the binding
146         raise EncryptedDatabaseError(
147             "database encryption requested but the 'sqlcipher3' package is not
installed"
148             ) from exc
149
150     conn = sqlcipher3.connect(path)
151     conn.execute(_raw_key_pragma(key)) # MUST be the first statement on the connection
152     try:
153         conn.execute("SELECT count(*) FROM sqlite_master") # forces key verification
154     except sqlcipher3.DatabaseError as exc:
155         conn.close()
156         raise EncryptedDatabaseError(
157             f"could not open encrypted database {Path(path).name}: wrong key, or not an
"
158             "encrypted muneem ledger"
159             ) from exc
160     conn.row_factory = sqlcipher3.Row
161     return conn
162
163
164     @contextmanager
165     def connect(db_path: Path | str, *, key: str | None = None) ->
Iterator[sqlite3.Connection]:
166         """Open a connection (commit on success, always close). Parent dirs created.
167
168         With ``key`` (a 64-hex-char SQLCipher key) the database is **encrypted at rest** via
169         SQLCipher. Without it, a plaintext stdlib :mod:`sqlite3` connection is used ? the
path for
170         ``:memory:`` and tests. A keyed open never silently falls back to plaintext (see
171         :func:`_open_encrypted`); the production ledger is opened via
:func:`connect_encrypted`.
172         """
173         path = str(db_path)
174         if path != ":memory:":
175             Path(path).parent.mkdir(parents=True, exist_ok=True)
176         if key is not None:
177             conn = _open_encrypted(path, key)
178             _harden_permissions(path)
179         else:
180             conn = sqlite3.connect(path)
181             conn.row_factory = sqlite3.Row
182             conn.execute("PRAGMA foreign_keys = ON")
183         try:
184             yield conn
185             conn.commit()
186         finally:
187             conn.close()
188
189
190     @contextmanager
191     def connect_encrypted(db_path: Path | str) -> Iterator[sqlite3.Connection]:
192         """Open the real on-disk ledger, encrypted with the keychain-managed key.
193
194         The production path: resolves (or mints, on first use) the 256-bit key from the OS
195         keychain and opens with SQLCipher. If ``db_path`` is a pre-existing *plaintext*
SQLite
196         file, refuses with a clear pointer to ``muneem db encrypt`` rather than a cryptic
197         "file is not a database" error.
198         """

```

```

199         from muneem.db_key import get_or_create_key
200
201         path = str(db_path)
202         if is_plaintext_sqlite(path):
203             raise EncryptedDatabaseError(
204                 f"{Path(path).name} is an unencrypted SQLite database; run 'muneem db
encrypt' "
205                 "to migrate it to an encrypted ledger"
206             )
207         with connect(path, key=get_or_create_key()) as conn:
208             yield conn
209
210
211     def encrypt_database(db_path: Path | str, key: str) -> int:
212         """Migrate a plaintext SQLite ledger to an encrypted one in place, via
213         ``sqlcipher_export``.
214
215         The original is preserved as ``<name>.plaintext.bak`` for the caller to verify and
216         then
217         securely delete (it still holds the cleartext data). Returns the number of user
218         tables
219         copied. Raises :class:`EncryptedDatabaseError` if ``db_path`` is not a plaintext
220         SQLite file.
221         """
222         import sqlcipher3
223
224         path = Path(db_path)
225         if not is_plaintext_sqlite(path):
226             raise EncryptedDatabaseError(
227                 f"{path.name} is not a plaintext SQLite database (already encrypted or
missing)"
228             )
229
230         enc_tmp = path.with_name(path.name + ".enc.tmp")
231         enc_tmp.unlink(missing_ok=True)
232         try:
233             src = sqlcipher3.connect(str(path)) # open the plaintext source (no key)
234             try:

```

5. user (2026-06-10T16:22:21.640Z)

```

1     """The ledger's encryption key ? minted once, kept in the OS keychain.
2
3     muneem's local ledger is **encrypted at rest** with SQLCipher (DESIGN.md §5.4 rule 4).
4     The 256-bit key is generated on first use and stored in the OS keychain via
5     :mod:`muneem.secrets` (Windows Credential Manager / macOS Keychain / Secret Service) ?
6     never on disk in plaintext, never in the repo, never logged.
7
8     It is handed to SQLCipher in **raw-key form** (``PRAGMA key = 'x'<64 hex>'``), which
uses
9     the 256-bit key directly and *skips* the passphrase KDF. That is correct here because
the
10    key is already full-entropy random; KDF stretching only buys security for low-entropy
human
11    passphrases (an opt-in we may add later ? see ROADMAP D.2 phased plan).
12    """
13
14    from __future__ import annotations
15
16    import secrets as _stdlib_secrets
17
18    from muneem import secrets as _keychain
19
20    # Keychain entry name (under the shared "muneem" service) holding the 64-hex-char DB
key.

```

```

21     _DB_KEY_NAME = "db-key"
22
23
24     def get_or_create_key() -> str:
25         """Return the ledger's 256-bit key as 64 hex chars, minting+storing one on first
use."""
26         existing = _keychain.get_secret(_DB_KEY_NAME)
27         if existing:
28             return existing
29         key = _stdlib_secrets.token_bytes(32).hex()
30         _keychain.set_secret(_DB_KEY_NAME, key)
31         return key
32
33
34     def peek_key() -> str | None:
35         """Return the stored key, or ``None`` if none has been minted yet. Never creates
one."""
36         return _keychain.get_secret(_DB_KEY_NAME)
37
38
39     def new_key() -> str:
40         """Generate a fresh random 256-bit key as 64 hex chars (not stored)."""
41         return _stdlib_secrets.token_bytes(32).hex()
42
43
44     def set_key(key: str) -> None:
45         """Overwrite the stored DB key ? used by key rotation, **after** the file has been
rekeyed."""
46         _keychain.set_secret(_DB_KEY_NAME, key)
47

```

6. assistant (2026-06-10T16:23:00.389Z)

Let me check the errors module, the secrets/keychain module, and tests covering rotate-key.

7. user (2026-06-10T16:23:01.166Z)

```

1     """muneem's exception hierarchy.
2
3     Every error muneem raises derives from :class:`MuneemError`, so callers can catch the
whole family
4     with one `except`. Two rules for messages on these exceptions, because they may end up
in logs:
5
6     1. **Only non-sensitive context.** A file *name* is acceptable (it's the user's own
machine), but
7         never raw statement text, passwords, balances, or account numbers. Use
8         :mod:`muneem.redaction` helpers to sanitise anything derived from a document.
9     2. **Chain the cause.** Raise with ``raise ParseError(...) from exc`` so the original
traceback is
10        available for local debugging while the muneem-level message stays clean.
11    """
12
13    from __future__ import annotations
14
15
16    class MuneemError(Exception):
17        """Base class for all errors raised by muneem."""
18
19
20    class ParseError(MuneemError):
21        """A document could not be parsed (corrupt PDF, unexpected layout, extraction
failure).
22
23        Carries only non-sensitive context (e.g. the file name and the failing stage) ?

```

```

never the
24         statement contents or the underlying library's message verbatim.
25         """
26
27
28     class UnlockError(MuneemError):
29         """A password-protected document could not be unlocked (no working password, bad
input)."""
30
31
32     class EncryptedDatabaseError(MuneemError):
33         """The encrypted ledger could not be opened ? wrong key, missing SQLCipher binding,
or a
34         plaintext file where an encrypted one was expected."""
35
36
37     class KeychainError(MuneemError):
38         """Storing a secret in the OS keychain failed (no backend, locked keychain, etc.).
39
40         The secret *name* is safe to include; never the value."""
41
42
43     class ConfigError(MuneemError):
44         """A user config file (e.g. password_rules.yaml) is malformed.
45
46         Messages must reference the file by name only ? never echo its contents, which may
hold
47         password material."""
48
49
50     class MigrationError(MuneemError):
51         """A schema migration could not be applied, or the on-disk schema diverged from the
52         migration history (e.g. the frozen baseline was edited without adding a
migration)."""
53

```

8. user (2026-06-10T16:23:02.117Z)

```

tests\test_db_crypto_ops.py:31:# --- rekey / rotation
-----
tests\test_db_crypto_ops.py:34:def test_rekey_changes_the_key_and_keeps_data(tmp_path):
tests\test_db_crypto_ops.py:37:     dbmod.rekey_database(p, KEY_A, KEY_B)
tests\test_db_crypto_ops.py:45:def test_rekey_with_wrong_current_key_raises(tmp_path):
tests\test_db_crypto_ops.py:49:     dbmod.rekey_database(p, KEY_B, KEY_A)
tests\test_db_crypto_ops.py:118:# --- CLI: db rotate-key / db backup
-----
tests\test_db_crypto_ops.py:127:def test_db_rotate_key_cli(tmp_path):
tests\test_db_crypto_ops.py:131:     res = runner.invoke(app, ["db", "rotate-key", "--db",
str(led)])
tests\test_db_crypto_ops.py:153:def
test_db_rotate_key_refuses_when_no_encrypted_ledger(tmp_path):
tests\test_db_crypto_ops.py:154:     res = runner.invoke(app, ["db", "rotate-key", "--db",
str(tmp_path / "nope.db")])

```

9. user (2026-06-10T16:23:06.507Z)

```

1         """Crypto operations on the encrypted ledger: key rotation, encrypted backup, migration
temp
2         cleanup, correctness at size, and the plaintext-refusal guidance ? plus their CLI
commands.
3
4         Offline, fabricated data only; the autouse conf test fixture keeps the real keychain
untouched.
5         """
6

```

```

7     import sqlite3
8
9     import pytest
10
11     pytest.importorskip("sqlcipher3")
12
13     from typer.testing import CliRunner # noqa: E402
14
15     from muneem import db as dbmod # noqa: E402
16     from muneem import db_key # noqa: E402
17     from muneem.cli import app # noqa: E402
18
19     runner = CliRunner()
20
21     KEY_A = "00112233445566778899aabbccddeeff00112233445566778899aabbccddeeff"
22     KEY_B = "ffeeddccbbaa99887766554433221100ffeeddccbbaa99887766554433221100"
23
24
25     def _seed_encrypted(path, key, n=3):
26         with dbmod.connect(path, key=key) as conn:
27             conn.execute("CREATE TABLE t (id INTEGER PRIMARY KEY, amount REAL)")
28             conn.executemany("INSERT INTO t (amount) VALUES (?)", [(float(i),) for i in
range(n)])
29
30
31     # --- rekey / rotation
-----
32
33
34     def test_rekey_changes_the_key_and_keeps_data(tmp_path):
35         p = tmp_path / "led.db"
36         _seed_encrypted(p, KEY_A, n=5)
37         dbmod.rekey_database(p, KEY_A, KEY_B)
38         with pytest.raises(dbmod.EncryptedDatabaseError): # old key no longer opens it
39             with dbmod.connect(p, key=KEY_A):
40                 pass
41         with dbmod.connect(p, key=KEY_B) as conn: # new key works, data intact
42             assert conn.execute("SELECT count(*) FROM t").fetchone()[0] == 5
43
44
45     def test_rekey_with_wrong_current_key_raises(tmp_path):
46         p = tmp_path / "led.db"
47         _seed_encrypted(p, KEY_A)
48         with pytest.raises(dbmod.EncryptedDatabaseError):
49             dbmod.rekey_database(p, KEY_B, KEY_A)
50
51
52     # --- encrypted backup
-----
53
54
55     def test_backup_is_encrypted_and_intact(tmp_path):
56         p = tmp_path / "led.db"
57         _seed_encrypted(p, KEY_A, n=4)
58         dest = tmp_path / "backup" / "led.bak.db"
59         dbmod.backup_database(p, dest, KEY_A)
60         assert dest.exists() and not dbmod.is_plaintext_sqlite(dest) # backup stays
encrypted
61         with dbmod.connect(dest, key=KEY_A) as conn:
62             assert conn.execute("SELECT count(*) FROM t").fetchone()[0] == 4
63         with pytest.raises(dbmod.EncryptedDatabaseError): # wrong key can't read the backup
64             with dbmod.connect(dest, key=KEY_B):
65                 pass
66
67

```

```

68     def test_backup_with_wrong_key_raises(tmp_path):
69         p = tmp_path / "led.db"
70         _seed_encrypted(p, KEY_A)
71         with pytest.raises(dbmod.EncryptedDatabaseError):
72             dbmod.backup_database(p, tmp_path / "b.db", KEY_B)
73
74
75     # --- migration cleanup + correctness at size + plaintext refusal
76     -----
77
78     def test_encrypt_database_cleans_temp_on_failure(tmp_path, monkeypatch):
79         p = tmp_path / "led.db"
80         con = sqlite3.connect(p)
81         con.execute("CREATE TABLE t (id INTEGER PRIMARY KEY)")
82         con.commit()
83         con.close()
84
85         def boom(*_a, **_k): # fail the post-export verification open
86             raise RuntimeError("boom during verify")
87
88         monkeypatch.setattr(dbmod, "_open_encrypted", boom)
89         with pytest.raises(RuntimeError):
90             dbmod.encrypt_database(p, KEY_A)
91         assert not (tmp_path / "led.db.enc.tmp").exists() # partial temp removed
92         assert dbmod.is_plaintext_sqlite(p) # original left untouched
93
94
95     def test_encrypted_ledger_handles_a_larger_table(tmp_path):
96         p = tmp_path / "led.db"
97         with dbmod.connect(p, key=KEY_A) as conn:
98             conn.execute("CREATE TABLE t (id INTEGER PRIMARY KEY, amount REAL)")
99             conn.executemany("INSERT INTO t (amount) VALUES (?)", [(float(i),) for i in
range(2000)])
100         with dbmod.connect(p, key=KEY_A) as conn:
101             total = conn.execute("SELECT SUM(amount) FROM t").fetchone()[0]
102             cnt = conn.execute("SELECT count(*) FROM t WHERE amount >= 1000").fetchone()[0]
103             assert cnt == 1000 and total == float(sum(range(2000)))
104
105
106     def test_connect_encrypted_plaintext_message_points_to_db_encrypt(tmp_path):
107         p = tmp_path / "led.db"
108         con = sqlite3.connect(p)
109         con.execute("CREATE TABLE t (id INTEGER)")
110         con.commit()
111         con.close()
112         with pytest.raises(dbmod.EncryptedDatabaseError) as ei:
113             with dbmod.connect_encrypted(p):
114                 pass
115         assert "muneem db encrypt" in str(ei.value) # actionable guidance, not a cryptic
error
116
117
118     # --- CLI: db rotate-key / db backup
119     -----
120
121     def _new_encrypted_ledger(path):
122         with dbmod.connect_encrypted(path) as conn:
123             dbmod.init_db(conn)
124             dbmod.insert_document(conn, source="file", sha256="s", status="parsed")
125
126
127     def test_db_rotate_key_cli(tmp_path):
128         led = tmp_path / "ledger.db"

```

```

129     _new_encrypted_ledger(led)
130     old = db_key.peek_key()
131     res = runner.invoke(app, ["db", "rotate-key", "--db", str(led)])
132     assert res.exit_code == 0, res.output
133     new = db_key.peek_key()
134     assert new and new != old
135     with dbmod.connect(led, key=new) as conn: # opens under the new key
136         assert dbmod.get_document_by_sha(conn, "s") is not None
137     with pytest.raises(dbmod.EncryptedDatabaseError): # old key no longer works
138         with dbmod.connect(led, key=old):
139             pass
140
141
142     def test_db_backup_cli(tmp_path):
143         led = tmp_path / "ledger.db"
144         _new_encrypted_ledger(led)
145         dest = tmp_path / "led.backup.db"
146         res = runner.invoke(app, ["db", "backup", str(dest), "--db", str(led)])
147         assert res.exit_code == 0, res.output
148         assert dest.exists() and not dbmod.is_plaintext_sqlite(dest)
149         with dbmod.connect(dest, key=db_key.peek_key()) as conn:
150             assert dbmod.get_document_by_sha(conn, "s") is not None
151
152
153     def test_db_rotate_key_refuses_when_no_encrypted_ledger(tmp_path):
154         res = runner.invoke(app, ["db", "rotate-key", "--db", str(tmp_path / "nope.db")])
155         assert res.exit_code == 1 and "db encrypt" in res.output
156

```

10. user (2026-06-10T16:23:07.050Z)

```

1     """Secret storage via the OS keychain.
2
3     OAuth tokens, cached statement passwords, and the DB encryption key are stored here
4     (Windows
5     Credential Manager / macOS Keychain / Secret Service), never in the repo, on disk in
6     plaintext, or
7     in logs. See DESIGN.md section 5.
8
9     External-call discipline (every keyring call is wrapped with attribution):
10    - **read** (:func:`get_secret`) is *graceful* ? a missing or unavailable backend returns
11      ``None``
12      (absence is a valid state; this also keeps headless CI happy).
13    - **write** (:func:`set_secret`) is *loud* ? a failure raises
14      :class:`~muneem.errors.KeychainError`,
15      because silently losing a write (e.g. the DB key) would be dangerous. The secret
16      *name* is
17      included for diagnosis; the value never is.
18    """
19
20    from __future__ import annotations
21
22    import keyring
23    from keyring.errors import KeyringError, PasswordDeleteError
24
25    from muneem.errors import KeychainError
26
27    _SERVICE = "muneem"
28
29
30    def get_secret(name: str) -> str | None:
31        """Return the stored secret, or ``None`` if absent or the keychain is
32        unavailable."""
33        try:
34            return keyring.get_password(_SERVICE, name)
35

```

```

29         except KeyringError:
30             return None
31
32
33     def set_secret(name: str, value: str) -> None:
34         """Store (or replace) a secret in the OS keychain. Raises KeychainError on
35 failure."""
36         try:
37             keyring.set_password(_SERVICE, name, value)
38         except KeyringError as exc:
39             # Name only ? never the value ? in the message (it may be a password or the DB
40 key).
41             raise KeychainError(f"could not store secret {name!r} in the OS keychain") from
42 exc
43
44     def delete_secret(name: str) -> None:
45         """Delete a secret if present; a no-op if it is not (or the keychain is
46 unavailable)."""
47         try:
48             keyring.delete_password(_SERVICE, name)
49         except (PasswordDeleteError, KeyringError):
50             pass

```

11. user (2026-06-10T16:24:15.704Z)

Structured output provided successfully

12. assistant (2026-06-10T16:24:19.950Z)

Verification complete. The finding is confirmed real (keychain-first key overwrite with an incomplete rollback in `db rotate-key`), with severity adjusted from critical to high given the rare trigger, the optional recovery-file mitigation, and re-ingestable source data.